1. Identificação do projeto

Projeto original: Sistema de Estoque e Loja
Branch: https://github.com/twolysm/trabalho-manutencao-software
Linguagem: Python
Tipo de aplicação: Sistema de linha de comando (CLI) para gerenciamento de estoque e operações de uma loja.

O projeto escolhido é um sistema simples de estoque e vendas desenvolvido em Python. A aplicação permite cadastrar e consultar produtos, controlar quantidades em estoque e realizar operações relacionadas às vendas, como aplicação de descontos, cálculo de impostos, frete, parcelamento, troco e validação de CPF.

2. Justificativa da escolha

O projeto foi escolhido por possuir tamanho pequeno, estrutura relativamente simples e linguagem conhecida pela equipe. Além disso, possui repositório público, histórico de desenvolvimento no Git, instruções de execução e testes automatizados. Outro fator importante foi a possibilidade de executar o sistema localmente no Windows sem necessidade de serviços externos, banco de dados ou configurações complexas. Apesar de ser um projeto pequeno, foram identificados pontos que possibilitam análise de manutenibilidade, refatoração e inclusão de uma nova funcionalidade.

3. Compreensão inicial do sistema

A aplicação é dividida em módulos. O arquivo main.py inicia o programa, enquanto menu.py controla a navegação entre o sistema da loja e o sistema de estoque. O arquivo estoque.py é responsável pelo gerenciamento dos produtos, e loja.py concentra as operações relacionadas às vendas. O arquivo operacoes.py contém funções auxiliares, como cálculos de desconto, imposto, frete, parcelamento e validação de CPF. Os testes automatizados originais estão concentrados principalmente nas funções de operacoes.py.

4. *Baseline*

Antes de qualquer alteração, o projeto original foi clonado e executado localmente. A aplicação iniciou corretamente e suas principais opções foram verificadas.

O commit utilizado como baseline foi:

Commit da *baseline*:
ba5d09c

A suíte de testes original foi executada utilizando:

python -m unittest discover

Foram executados 16 testes, todos concluídos com sucesso.

Resultado da *baseline*: aplicação executada com sucesso e 16 de 16 testes aprovados.

Por se tratar de uma aplicação Python, não foi necessária uma etapa específica de compilação ou build antes da execução.

5. Análise de manutenibilidade

Durante a análise do código foram identificados cinco pontos principais de manutenibilidade.

5.1 Função com muitas responsabilidades

A função registrar_compra(), localizada em loja.py, concentrava diversas atividades do processo de venda, como montagem do carrinho, cálculo dos valores, validação de CPF, processamento do pagamento e atualização do estoque. Essa concentração dificultava a leitura e futuras alterações, pois diferentes responsabilidades estavam reunidas em um único método.

5.2 Uso de estado global

O estoque é mantido por meio de uma variável global contendo os produtos. Essa estrutura é acessada por diferentes partes do sistema, aumentando o acoplamento e dificultando a realização de testes isolados ou mudanças futuras na forma de armazenamento.

5.3 Tratamento genérico de exceções

Foram encontrados pontos utilizando except Exception, que captura erros de forma muito ampla. Esse tratamento pode dificultar a identificação da causa real de falhas e esconder erros inesperados.

5.4 Valores fixos nas regras de negócio

Alguns valores relacionados a imposto, frete e limite de estoque estão diretamente inseridos nas funções. Caso essas regras sejam alteradas futuramente, será necessário localizar os valores no código, aumentando o risco de inconsistências.

5.5 Cobertura de testes concentrada

Os testes originais estavam concentrados principalmente no módulo operacoes.py. Outras partes importantes, como o processo de compra e controle do histórico de vendas, não possuíam cobertura equivalente.

6. Refatoração realizada

Entre os problemas identificados, foi escolhida para refatoração a função registrar_compra(), devido à quantidade de responsabilidades concentradas nela. A técnica utilizada foi Extract Method. As diferentes etapas do processo foram separadas em funções menores, responsáveis pela montagem do carrinho, cálculo dos valores da compra, solicitação de CPF,

processamento do pagamento e atualização do estoque. A escolha dessa técnica teve como objetivo melhorar a legibilidade e facilitar futuras modificações sem alterar o comportamento esperado do sistema. Após a refatoração, os testes existentes foram executados novamente para verificar se o comportamento anterior havia sido preservado.

Commit correspondente à refatoração:
e14c57d

7. Solicitação de manutenção perfectiva

Como evolução funcional, foi proposta a implementação de um histórico de vendas. O sistema original realizava a venda e atualizava o estoque, porém não permitia consultar posteriormente as vendas realizadas. A nova funcionalidade tem como objetivo registrar as compras concluídas e disponibilizar essas informações para consulta. O histórico deve registrar os produtos comprados, as respectivas quantidades, preços e o valor total da venda. Apenas compras concluídas com sucesso devem ser registradas.

8. Análise de impacto

Antes da implementação, foi previsto que a alteração afetaria principalmente o arquivo loja.py, pois nele ocorre o processo de compra e é apresentado o menu do sistema da loja. Também foi prevista a necessidade de novos testes automatizados para verificar o registro e a consulta das vendas. Não foram previstas alterações relevantes nos módulos estoque.py, operacoes.py e menu.py, nem a instalação de novas bibliotecas ou dependências externas. O principal risco identificado foi registrar uma venda antes da conclusão correta da compra ou interferir no processo já existente de atualização do estoque.

9. Implementação da manutenção

Foi implementada a funcionalidade de histórico de vendas. Após a conclusão de uma compra, as informações da venda são registradas em uma estrutura mantida durante a execução do programa. Também foi adicionada ao Sistema da Loja a opção “Histórico de vendas”, permitindo visualizar as vendas realizadas.

A implementação foi integrada ao fluxo existente de compra sem necessidade de novas bibliotecas externas.

Commit correspondente à nova funcionalidade:
e14c57d

10. Verificação e testes de regressão

Na *baseline*, o projeto possuía 16 testes automatizados, todos aprovados. Após a implementação da nova funcionalidade, foram adicionados 4 novos testes relacionados ao histórico de vendas. Ao executar novamente a suíte completa, foram realizados:

20 testes executados
20 testes aprovados
Resultado: OK

Também foi realizado um teste manual no sistema. Uma compra foi concluída e, em seguida, consultada pela opção de histórico de vendas. O sistema apresentou corretamente os produtos, quantidades, preços e valor total da venda.

Não foram identificadas regressões pelos testes disponíveis.

Commit correspondente aos novos testes:
c19d823

11. Impacto previsto x impacto observado

O impacto observado após a implementação foi próximo ao que havia sido previsto. A principal alteração ocorreu no módulo relacionado às operações da loja, onde foi integrada a funcionalidade de registro e consulta do histórico. Também foram adicionados testes específicos para a nova funcionalidade. Não foi necessária a inclusão de novas dependências nem alterações significativas nos módulos de estoque e operações matemáticas. Os testes anteriores continuaram funcionando após a modificação, indicando que a manutenção não introduziu regressões identificáveis pela suíte de testes.

12. Reflexão sobre o trabalho

O desenvolvimento da atividade permitiu compreender na prática o processo de manutenção de um software criado por terceiros. A principal dificuldade inicial foi entender a organização do projeto e identificar quais partes poderiam ser modificadas sem comprometer seu funcionamento. O estabelecimento da *baseline* foi importante para possuir uma referência antes das alterações. A execução dos testes antes e depois da refatoração e da manutenção também permitiu verificar se as mudanças introduziram efeitos indesejados. A refatoração mostrou como a divisão de uma função com muitas responsabilidades pode facilitar a compreensão e manutenção do código. Já a implementação do histórico de vendas permitiu aplicar a análise de impacto antes da alteração e posteriormente comparar o impacto previsto com o realmente observado.